Relatório Flappy Bird

1. Membros do grupo

- Felipe Castilho - a047152
- Juliana Moreira - a047188
- Marta Vieira - a046756

2. Descrição do jogo

2.1 Nome do jogo:

Flappy Bird

2.2 Descrição

Este jogo é baseado no jogo Flappy Bird. Basicamente o objetivo é passar o maior número de canos possível, no espaço entre os dois, onde é possível usar somente a spacebar que o faz voar.

2.3 Mecânicas do jogo

- **Objetivo**: Passar pelo maior número possível de pipes sem colidir
- **Controlos**: Barra de espaço para fazer o pássaro voar
- **Física**: Simulada com gravidade e velocidade (voo e queda)
- **Pontuação**: 1 ponto por cada pipe ultrapassado com sucesso
- **Best Score**: Sistema que guarda a melhor pontuação após cada jogo

3. Ferramentas Utilizadas

3.1 Linguagem e Versão

- **Java**: Amazon Corretto 1.8
- **IDE**: IntelliJ IDEA / Eclipse

3.2 Biblioteca

- **Lanterna 3.1.2**

3.3 Testes

- **JUnit 5 (Jupiter)**: para realização de testes

3.4 Outras Dependências

- **Java Sound API**: para efeitos sonoros

4. Funcionalidades

4.0 Inicialização do Jogo

Código em App.java:

```
public class App {
    public static void main(String[] args) {
        Game game = new Game();  // Cria instância principal do jogo
        game.run();              // Inicia loop principal
    }
}
```

Código em Game.java:

```
Game() {
    try {
        // Configuração do terminal usando Lanterna
        DefaultTerminalFactory terminalFactory = new DefaultTerminalFactory();
        terminalFactory.setInitialTerminalSize(new TerminalSize(gameWidth, 45));

        Terminal terminal = terminalFactory.createTerminal();

        // Centraliza janela se for SwingTerminalFrame
        if (terminal instanceof SwingTerminalFrame) {
            SwingTerminalFrame terminalFrame = (SwingTerminalFrame) terminal;
            Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
            int x = (screenSize.width - terminalFrame.getWidth()) / 2;
            int y = ((screenSize.height - terminalFrame.getHeight()) / 2) - 13;
            terminalFrame.setLocation(x, y);
        }

        // Cria e inicia screen
        screen = new TerminalScreen(terminal);
        screen.setCursorPosition(null);
        screen.startScreen();

        // Obtém tamanho do terminal e cria Arena
        terminalSize = screen.getTerminalSize();
        arena = new Arena(gameWidth, terminalSize.getRows(), this);  // Passa referência de

        // Cria e inicia thread de input
        c = new Controller(this, screen);
        t = new Thread(c);
        t.start();

        finish = false;
        gameState = 0;  // Começa no menu
```

```
        // Inicializa sistema de sons
        this.SoundEffects = new SoundEffects();

    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

4.1 Gameplay

Sistema de Reset e Reinício Código em Arena.java:

```
public void start() {  // Chamado quando inicia ou reinicia
    // Reposiciona bird no início
    bird.setPosition(new Position(width/3, height/2));
    bird_vel = 0.0;  // Reseta velocidade

    // Reseta estado visual do bird
    bird.setState(SpritesBird.BIRDY);

    // Reseta flags de morte
    birdDied = false;
    deadVel = 0;

    // Limpa todos os pipes
    pipes.clear();

    // Reseta pontuação
    score = 0;
}

public void clean() {  // Limpa pipes (usado ao sair)
    pipes.clear();
}
```

Código em Arena.java - Atualização dinâmica da Altura:

```
public void updateTerminalHeight(int newTerminalHeight) {
    this.viewMaxHeight = Math.min(newTerminalHeight, maxArenaHeight);
    this.height = viewMaxHeight;
    this.walls = createWalls();  // Recria paredes com nova altura

    // Ajusta posição do bird se necessário
    if (bird.getY() >= height - 1) {
        bird.setPosition(new Position(bird.getX(), height / 2));
    }
```

```
}
```

Sistema de Física do Bird

- **Gravidade**: Aceleração constante para baixo (0.3)
- **Salto**: "voa" ao pressionar espaço (-1.4)
- **Velocidade Máxima de Queda**: Limitada a 2.5 para evitar quedas demasiado rápidas
- **Movimento Contínuo**: Atualização da posição a cada frame baseada na velocidade atual

Código em Arena.java:

```
private double bird_vel = 0.0; //velocidade do passaro pa cima/baixo
private final double gravity = 0.3; //gravidade a cada run/loop
private final double jump = -1.4; //velocidade após spacebar
private final double maxFallSpeed = 2.5; //referencia de max para nao ter velocidade mt alt

public void updateBird() {
    if (birdDied) {
        deadVel += gravity;
        if (deadVel > maxFallSpeed) {
            deadVel = maxFallSpeed;
        }
        int novoY = (int)(bird.getY() + deadVel);
        bird.setPosition(new Position(bird.getX(), novoY));
    } else {
        bird_vel += gravity;
        if (bird_vel > maxFallSpeed) {
            bird_vel = maxFallSpeed;
        }
        int novoY = (int) (bird.getY() + bird_vel);
        bird.setPosition(new Position(bird.getX(), novoY));
    }
}
```

Sistema de Pipes

- **Geração**: Pipes são gerados continuamente à direita do ecrã
- **Gap Aleatório**: Posição vertical do gap é randomizada entre limites seguros
- **Movimento**: Pipes movem-se da direita para a esquerda
- **Remoção**: Pipes que saem do ecrã são removidos
- **Distância**: espaço de 25 píxeis entre pipes

Código em Arena.java:

```
private final int pipe_distance = 25; //distância entre cada pipe na geração
```

```
private final int pipe_gap = 13;

public void updatePipes() {
    List<Pipe> PipeRemover = new ArrayList<>();

    for (Pipe p : pipes) {
        p.moveLeft();
        giveScore(p);

        if (p.getX() + pipe_width < 0) {
            PipeRemover.add(p);
        }
    }

    pipes.removeAll(PipeRemover);

    if (pipes.isEmpty() || pipes.get(pipes.size() - 1).getX() <= nextPipeX) {
        int gapStart = random.nextInt(height - pipe_gap - 5) + 3;
        pipes.add(new Pipe(width - 6, gapStart, pipe_width, pipe_gap, height));
    }
}
```

Código em Pipe.java:

```
public void moveLeft() {
    position = new Position(position.getX() - 1, position.getY());
}
```

Sistema de Colisões

- **Hitbox**: Bird tem uma hitbox de 8×4 caracteres
- **Deteção de colisão com Pipes**: Verifica sobreposição entre limites do bird e dos pipes
- **Deteção de colisão com Paredes**: Verifica limites superior e inferior da arena
- **Resposta à Colisão**:
  - Mudança de sprite para "DEADBIRDY"
  - Reprodução de som de impacto
  - Animação de queda até ao chão
  - Transição para ecrã de game over

Código em Arena.java:

```
public void checkCollision() {
    if (!birdDied) {
        if (bird.getY() <= 0 || bird.getY() >= height - 1) {
            bird.setState(SpritesBird.DEADBIRDY);
            SoundEffects.playHit();
```

```
            startFall();
            SoundEffects.playDies();
            return;
        }

        for (Pipe p : pipes) {
            if (p.collidePipe(bird)) {
                bird.setState(SpritesBird.DEADBIRDY);
                SoundEffects.playHit();
                startFall();
                SoundEffects.playDies();
                return;
            }
        }
    }
}
```

Código em Pipe.java:

```
public boolean collidePipe(Bird bird) {
    int birdLeft   = bird.getLeftX();
    int birdRight  = bird.getRightX();
    int birdStart  = bird.getStartY();
    int birdEnd    = bird.getEndY();

    if (birdRight < getX() || birdLeft >= getX() + width) {
        return false;
    }

    if (birdStart < gapStart || birdEnd >= gapStart + gap) {
        return true;
    }

    return false;
}
```

Código em Bird.java - Sistema de Hitbox:

```
private static final int bird_width = 8;
private static final int bird_height = 4;

//para a hitbox
public int getStartY() {
    return getY();
}

public int getEndY() {
    return getY() + bird_height - 1;
```

```
}

public int getLeftX() {
    return getX() - (bird_width / 2);
}

public int getRightX() {
    return getLeftX() + bird_width - 1;
}

public int getHeight() {
    return bird_height;
}
```

Interação entre múltiplas classes na deteção de colisão (Arena -> Pipe -> Bird):

```
// Em Arena.java - checkCollision()
public void checkCollision() {
    if (!birdDied) {
        // Colisão com paredes (Arena verifica limites)
        if (bird.getY() <= 0 || bird.getY() >= height - 1) {
            bird.setState(SpritesBird.DEADBIRDY);  // Bird muda estado
            SoundEffects.playHit();                 // SoundEffects reproduz som
            startFall();                            // Arena inicia queda
            SoundEffects.playDies();
            return;
        }

        // Colisão com pipes (Arena itera sobre pipes, Pipe verifica colisão com Bird)
        for (Pipe p : pipes) {
            if (p.collidePipe(bird)) {              // Pipe usa métodos de hitbox do Bird
                bird.setState(SpritesBird.DEADBIRDY);
                SoundEffects.playHit();
                startFall();
                SoundEffects.playDies();
                return;
            }
        }
    }
}
```

Sistema de Pontuação

- **Score**: Atualizado quando bird passa completamente um pipe
- **Prevenção de Duplicação**: Sistema de flags/marcação para garantir que cada pipe só dá 1 ponto

- **Best Score**: Mantém registo da melhor pontuação alcançada
- **Display**: Score visível no canto superior esquerdo durante o jogo
- **Interface Scoring**: Usada para implementar sistema de pontuação

Código em Arena.java - Integração do sistema de pontuação:

```
private void giveScore(Pipe pipe) {
    // Arena chama método da interface Scoring implementada por Pipe
    // Verifica se bird passou o pipe e se ainda não foi pontuado
    if (pipe.checkAndScore(bird, pipe.getX(), pipe_width) && !birdDied) {
        score += pipe.getScoreValue();  // Pipe retorna valor (1 ponto)
        SoundEffects.playScore();       // Reproduz som de pontuação
    }
}

private void updateBestScore() {
    if (score > bestScore) {
        bestScore = score;
    }
}
```

Código em Pipe.java - Implementação da interface Scoring:

```
public class Pipe extends GameComplements implements Scoring {
    private boolean scored; //inic. false

    @Override
    public boolean isScored() {
        return scored;
    }

    @Override
    public void setAsScored(boolean scored) {
        this.scored = scored;
    }

    @Override
    public int getScoreValue() {
        return 1;  //return 1 = 1 ponto cada pipe
    }
}
```

Código em Scoring.java (Interface):

```
public interface Scoring {
    boolean isScored();
    void setAsScored(boolean scored);
    int getScoreValue();
```

```
    default boolean checkAndScore(Bird bird, int pipeX, int pipeWidth) {
        if (isScored())
            return false;

        int birdX = bird.getX();
        int passedPipe = pipeX + pipeWidth;

        if (birdX > passedPipe && !isScored()) {
            setAsScored(true);
            return true;
        }

        return false;
    }
}
```

Código em Arena.java:

```
private void giveScore(Pipe pipe) {
    if (pipe.checkAndScore(bird, pipe.getX(), pipe_width) && !birdDied) {
        score += pipe.getScoreValue();
        SoundEffects.playScore();
    }
}

private void updateBestScore() {
    if (score > bestScore) {
        bestScore = score;
    }
}
```

Código em Pipe.java:

```
public class Pipe extends GameComplements implements Scoring {
    private boolean scored;

    @Override
    public boolean isScored() {
        return scored;
    }

    @Override
    public void setAsScored(boolean scored) {
        this.scored = scored;
    }

    @Override
    public int getScoreValue() {
```

```
            return 1;  //return 1 = 1 ponto cada pipe
        }
    }
```

4.2 Sistema de Estados do Jogo

O jogo possui três estados principais: - **Estado 0 (Menu)**: Tela inicial com título e instruções - **Estado 1 (Gameplay)**: Jogo em execução - **Estado 2 (Game Over)**: Tela de fim de jogo com pontuação e instruções

Código em Game.java - Gestão de estados e ciclo principal:

```
private int gameState;

public void run() {
    while (!finish) {
        try {
            draw();  // Desenha frame atual

            if (gameState == 1) {  // Apenas atualiza se estiver jogando
                arena.updateBird();    // Arena atualiza física do bird
                arena.updatePipes();  // Arena atualiza posição dos pipes
                arena.checkCollision(); // Arena verifica colisões
            }

            Thread.sleep(40);  // ~25 FPS
        } catch(IOException | InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
}

public void processKey(KeyStroke key) {
    if (key.getKeyType() == KeyType.Character && key.getCharacter() == ' ') {
        if(gameState == 0) {  // Menu → Inicia jogo
            SoundEffects.playStarting();
            arena.start();
            gameState = 1;
        }
        else if (gameState == 1) {  // Jogando → Bird salta
            arena.processKey(key);
        }
        else if (gameState == 2) {  // Game Over → Reinicia
            SoundEffects.playStarting();
            arena.start();
            gameState = 1;
        }
```

```
    }
    else if (key.getKeyType() == KeyType.Character && key.getCharacter() == 'e') {
        if (gameState == 2) {  // Game Over → Volta ao menu
            gameState = 0;
        }
    }
}

public void died() {  // Chamado por Arena quando bird morre
    arena.clean();
    gameState = 2;
}
```

Código em Game.java - Renderização por estado:

```
public void draw() throws IOException {
    // Resize dinâmico do terminal
    TerminalSize newTerminalSize = screen.doResizeIfNecessary();
    if(newTerminalSize != null) {
        terminalSize = newTerminalSize;
        StartOfGameScreenX = (terminalSize.getColumns() - gameWidth) / 2;
        int gameHeight = Math.min(terminalSize.getRows(), max_height_arena);
        StartOfGameScreenY = (terminalSize.getRows() - gameHeight) / 2;
        arena.updateTerminalHeight(gameHeight);  // Atualiza Arena com nova altura
    }

    screen.clear();

    if(gameState == 0) {  // Menu
        TextGraphics graphics = screen.newTextGraphics();
        // Desenha título usando enum SpritesGame
        String[] draw = SpritesGame.TITLE.getSprite();
        // ... código de renderização do título
    }

    if (gameState == 1) {  // Jogando
        // Arena desenha todos os elementos (pipes, walls, bird, score)
        arena.draw(screen.newTextGraphics(), StartOfGameScreenX, StartOfGameScreenY);
    }

    else if (gameState == 2) {  // Game Over
        TextGraphics graphics = screen.newTextGraphics();
        // Desenha "GAME OVER" usando enum SpritesGame
        String[] draw = SpritesGame.ENDING.getSprite();
        // Mostra best score da Arena
        String bestScore = "Best Score: " + arena.getBestScore();
        // ... código de renderização
```

```
    }

    screen.refresh();
}
```

4.3 Sistema de Renderização (game drawing)

- **Sprites ASCII**: Pássaro e elementos do jogo renderizados com caracteres ASCII
- **Background do bird dinâmico**: adapta-se ao ambiente (pipes, céu, paredes)
- **Centralização Dinâmica**: Jogo centraliza-se automaticamente no terminal
- **Resize Responsivo**: Suporta redimensionamento do terminal

Código em Arena.java - Renderização completa da arena:

```
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
    // Fundo azul da arena
    graphics.setBackgroundColor(TextColor.Factory.fromString("#4FC3F7"));
    graphics.fillRectangle(new TerminalPosition(StartOfGameScreenX, StartOfGameScreenY),
                           new TerminalSize(width, height), ' ');

    // Desenha todos os pipes (itera sobre lista)
    for (Pipe pipe : pipes) {
        pipe.draw(graphics, StartOfGameScreenX, StartOfGameScreenY);
    }

    // Desenha todas as paredes (itera sobre lista)
    for (Wall wall : walls) {
        wall.draw(graphics, StartOfGameScreenX, StartOfGameScreenY);
    }

    // Desenha bird (passa lista de pipes para cálculo de background dinâmico)
    bird.draw(graphics, StartOfGameScreenX, StartOfGameScreenY, pipes, height);

    // Desenha score
    graphics.setBackgroundColor(TextColor.Factory.fromString("#FFFFFF"));
    graphics.setForegroundColor(TextColor.Factory.fromString("#000000"));
    graphics.putString(new TerminalPosition(StartOfGameScreenX + 2, StartOfGameScreenY + 2),
                       "Score: " + score);
}
```

Código em Bird.java - Renderização com background dinâmico:

```
// Override do método abstrato de GameComplements
@Override
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
```

```
    // Chama versão com mais parâmetros (necessário para o override)
    draw(graphics, StartOfGameScreenX, StartOfGameScreenY, null, 0);
}

// Overload: versão com parâmetros adicionais para background dinâmico
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY,
                 List<Pipe> pipes, int arenaHeight) {
    if(end) return;

    String[] draw = birdState.getSprite();  // Obtém sprite do enum SpritesBird
    int midX = draw[0].length()/2;  // Centro do bird

    for(int y = 0; y < draw.length; y++) {
        for(int x = 0; x < draw[y].length(); x++) {
            char c = draw[y].charAt(x);
            int px = x - midX;
            int py = y;

            // Calcula cor de fundo dinâmica baseada na posição (interage com pipes e arena)
            TextColor bgColor = getColorAt(position.getX() + px, position.getY() + py,
                                           pipes, arenaHeight);

            if (c == '>') {  // Bico - laranja
                graphics.setForegroundColor(TextColor.Factory.fromString("#FF7034"));
                graphics.setBackgroundColor(bgColor);
                graphics.enableModifiers(SGR.BOLD);
                graphics.putString(new TerminalPosition(StartOfGameScreenX + (int)(position.
                                                        StartOfGameScreenY + (int)(position.g
                                   String.valueOf(c));
            } else if(c == 'O') {  // Olho - preto
                graphics.setForegroundColor(TextColor.Factory.fromString("#000000"));
                graphics.setBackgroundColor(bgColor);
                graphics.enableModifiers(SGR.BOLD);
                graphics.putString(new TerminalPosition(StartOfGameScreenX + (int)(position.
                                                        StartOfGameScreenY + (int)(position.g
                                   String.valueOf(c));
            } else if(c != '*') {  // Corpo - azul escuro
                graphics.setForegroundColor(TextColor.Factory.fromString("#00008B"));
                graphics.setBackgroundColor(bgColor);
                graphics.enableModifiers(SGR.BOLD);
                graphics.putString(new TerminalPosition(StartOfGameScreenX + (int)(position.
                                                        StartOfGameScreenY + (int)(position.g
                                   String.valueOf(c));
            }
        }
    }
```

```
}

private TextColor getColorAt(double x, double y, List<Pipe> pipes, int arenaHeight) {
    // Verifica se está nas paredes (usa altura da arena)
    if (y <= 0 || y >= arenaHeight - 1) {
        return TextColor.Factory.fromString("#000000"); // preto
    }

    // Verifica se está sobre um pipe (itera sobre lista de pipes)
    for (Pipe pipe : pipes) {
        int pipeX = pipe.getX();
        if (x >= pipeX && x < pipeX + 6) {  // Área horizontal do pipe
            int gapStart = pipe.getGapStart();
            int gapEnd = gapStart + pipe.getGap();
            if (y < gapStart || y >= gapEnd) {  // Fora do gap = está no pipe
                return TextColor.Factory.fromString("#008000"); // verde
            }
        }
    }
    return TextColor.Factory.fromString("#4FC3F7"); // azul céu
}
```

Código em Pipe.java - Renderização dos pipes:

```
@Override
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
    graphics.setForegroundColor(TextColor.Factory.fromString("#008000")); // verde pipe
    graphics.setBackgroundColor(TextColor.Factory.fromString("#4FC3F7")); // azul fundo

    // Pipe de cima (do topo até ao gap)
    for (int y = 1; y < gapStart; y++) {
        for (int c = 0; c < width; c++) {
            int Draw = StartOfGameScreenX + getX() + c;
            if (Draw >= StartOfGameScreenX) {
                graphics.putString(new TerminalPosition(Draw, y + StartOfGameScreenY), " ");
            }
        }
    }

    // Pipe de baixo (fim do gap até ao fim da altura da arena)
    for (int y = gapStart + gap; y < arenaHeight - 1; y++) {
        for (int c = 0; c < width; c++) {
            int Draw = StartOfGameScreenX + getX() + c;
            if (Draw >= StartOfGameScreenX) {
                graphics.putString(new TerminalPosition(Draw, y + StartOfGameScreenY), " ");
            }
        }
```

```
    }
}
```

Código em Wall.java - Renderização das paredes:

```
public class Wall extends GameComplements {
    private String s;

    Wall(int x, int y, String s) {
        super(x, y);  // Chama construtor de GameComplements
        this.s = s;
    }

    @Override
    public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY)
        graphics.setForegroundColor(TextColor.Factory.fromString("#000000")); // preto
        graphics.setBackgroundColor(TextColor.Factory.fromString("#000000"));
        graphics.enableModifiers(SGR.BOLD);
        graphics.putString(new TerminalPosition(StartOfGameScreenX + (int) position.getX(),
                                                StartOfGameScreenY + (int) position.getY()),
    }
}
```

4.4 Sistema de Sons

- **Efeitos Sonoros**: 5 sons diferentes para ações diferentes
  - `wing.wav`: Som de salto/voo
  - `point.wav`: Som de pontuação
  - `hit.wav`: Som de colisão
  - `swoosh.wav`: Som de transição do início
  - `die.wav`: Som de morte

Código em SoundEffects.java - Gestão de sons:

```
public class SoundEffects {
    // Composição: SoundEffects contém múltiplos objetos Sound
    private Sound jumpSE;
    private Sound scoreSE;
    private Sound hitSE;
    private Sound startingSE;
    private Sound diesSE;

    public SoundEffects() {
        jumpSE = new Sound();
        jumpSE.setFile("sounds/wing.wav");

        scoreSE = new Sound();
        scoreSE.setFile("sounds/point.wav");
```

```
        hitSE = new Sound();
        hitSE.setFile("sounds/hit.wav");

        startingSE = new Sound();
        startingSE.setFile("sounds/swoosh.wav");

        diesSE = new Sound();
        diesSE.setFile("sounds/die.wav");
    }

    public void playJump() {
        try {
            jumpSE.play();
        } catch (Exception e) {
            // Falha silenciosa se som não disponível
        }
    }

    public void playScore() {
        try {
            scoreSE.play();
        } catch (Exception e) {}
    }

    public void playHit() {
        try {
            hitSE.play();
        } catch (Exception e) {}
    }

    public void playStarting() {
        try {
            startingSE.play();
        } catch (Exception e) {}
    }

    public void playDies() {
        try {
            diesSE.play();
        } catch (Exception e) {}
    }
}
```

Código em Sound.java - Classe base para reprodução de áudio:

```
import javax.sound.sampled.*;
```

```java
import java.io.File;

public class Sound {
    Clip clip;

    public void setFile(String soundFileName) {
        try {
            File file = new File(soundFileName);
            AudioInputStream sound = AudioSystem.getAudioInputStream(file);
            clip = AudioSystem.getClip();
            clip.open(sound);
        } catch (Exception e) {
            System.err.println("Erro ao carregar som " + e.getMessage());
        }
    }

    public void play() {
        clip.setFramePosition(0);  // Reinicia do início
        clip.start();
    }
}
```

Integração de sons em Arena.java:

```java
// Arena cria instância de SoundEffects
private SoundEffects SoundEffects;

Arena(int width, int height, Game game) {
    // ...
    this.SoundEffects = new SoundEffects();
}

// Sons são reproduzidos em diferentes momentos:
public void processKey(KeyStroke key) {
    if (key.getKeyType() == KeyType.Character && key.getCharacter() == ' ') {
        bird_vel = jump;
        SoundEffects.playJump();  // Som de salto
    }
}

private void giveScore(Pipe pipe) {
    if (pipe.checkAndScore(bird, pipe.getX(), pipe_width) && !birdDied) {
        score += pipe.getScoreValue();
        SoundEffects.playScore();  // Som de pontuação
    }
}
```

```
public void checkCollision() {
    if (!birdDied) {
        if (bird.getY() <= 0 || bird.getY() >= height - 1) {
            bird.setState(SpritesBird.DEADBIRDY);
            SoundEffects.playHit();   // Som de impacto
            startFall();
            SoundEffects.playDies();  // Som de morte
            return;
        }
        // ... mesmo padrão para colisão com pipes
    }
}
```

4.5 Sistema de Controlos

- **Thread Separada**: Input é processado em thread separada para não bloquear o jogo
- **Teclas Suportadas**:
  - Espaço: Iniciar jogo / Fazer pássaro voar / Reiniciar
  - 'Q': Sair do jogo
  - 'E': Voltar ao menu (após game over)

Código em Controller.java - Thread de input:

```
public class Controller implements Runnable {
    private Game game;
    private Screen screen;
    private boolean finish;

    public Controller(Game game, Screen screen) {
        this.game = game;
        this.screen = screen;
        finish = false;
    }

    @Override
    public void run() {
        while (!finish) {
            KeyStroke key = null;
            try {
                key = screen.readInput();  // Lê input do terminal (bloqueante)
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
            game.processKey(key);  // Envia para Game processar
        }
    }
```

```
    public void finish() {
        finish = true;  // Permite terminar thread graciosamente
    }
}
```

Implementação da interface Runnable: - `Controller` implementa `Runnable` para poder ser executado em thread separada - Permite que input seja processado de forma assíncrona sem bloquear o loop principal do jogo - Thread é iniciada em `Game()` e terminada através do método `finish()`

Código em Game.java - Inicialização do Controller:

```
Game() {
    // ...
    c = new Controller(this, screen);  // Cria controller com referência a Game e Screen
    t = new Thread(c);                  // Cria thread com controller
    t.start();                          // Inicia thread de input
    // ...
}

public void processKey(KeyStroke key) {
    // Game processa teclas e delega para Arena quando necessário
    if (key.getKeyType() == KeyType.Character && key.getCharacter() == ' ') {
        if (gameState == 1) {
            arena.processKey(key);  // Delega salto para Arena
        }
        // ... outros estados
    }
}
```

Código em Arena.java - Processamento do input do bird:

```
public void processKey(KeyStroke key) {
    if (key.getKeyType() == KeyType.Character && key.getCharacter() == ' ') {
        bird_vel = jump;  // Aplica força de salto
        SoundEffects.playJump();  // Reproduz som
    }
}
```

Código em Arena.java - Métodos de acesso para testes:

```
// Métodos adicionados para permitir testes unitários
public Bird getBird() {
    return bird;  // Retorna referência ao bird para testes
}

public List<Pipe> getPipes() {
    return new ArrayList<>(pipes);  // Retorna cópia para segurança (evita modificação exte
```

```
}

public boolean isBirdDead() {
    return birdDied;  // Permite verificar estado do bird em testes
}

public int getScore() {
    return score;  // Acesso ao score atual
}

public int getBestScore() {
    return bestScore;  // Acesso ao melhor score
}
```

4.6 Criação e Gestão de Elementos do Jogo

Código em Arena.java - Criação de paredes:

```
private List<Wall> createWalls() {
    List<Wall> walls = new ArrayList<>();

    // Paredes horizontais (topo e fundo)
    for (int c = 0; c < width; c++) {
        walls.add(new Wall(c, 0, "-"));              // Topo
        walls.add(new Wall(c, height - 1, "-"));       // Fundo
    }

    // Paredes verticais (esquerda e direita)
    for (int r = 1; r < height - 1; r++) {
        walls.add(new Wall(0, r, "|"));                // Esquerda
        walls.add(new Wall(width - 1, r, "|"));        // Direita
    }

    return walls;
}
```

Código em Arena.java - Criação de pipes:

```
private List<Pipe> createPipes() {
    List<Pipe> pipes = new ArrayList<>();

    // Gera gap aleatório entre limites seguros
    int gapStart = random.nextInt(height - pipe_gap - 5) + 3;

    // Cria pipe inicial à direita do ecrã
    pipes.add(new Pipe(width - 6, gapStart, pipe_width, pipe_gap, height));
```

```
    return pipes;
}

Arena(int width, int height, Game game) {
    this.width = width;
    this.viewMaxHeight = Math.min(height, maxArenaHeight);
    this.height = this.viewMaxHeight;
    this.game = game;

    bird = new Bird(width/3, this.height/2);  // Bird no meio-esquerda
    this.walls = createWalls();                  // Cria paredes
    this.pipes = createPipes();                 // Cria pipe inicial
    this.nextPipeX = (width - 6) - pipe_distance;  // Posição do próximo pipe
    this.score = 0;
    this.SoundEffects = new SoundEffects();
}
```

Código em GameComplements.java - Classe base abstrata:

```
public abstract class GameComplements {
    protected Position position;  // Posição compartilhada por todas as subclasses

    public GameComplements(int x, int y) {
        this.position = new Position(x, y);  // Cria Position com coordenadas
    }

    public int getX() {
        return (int) position.getX();  // Acesso à posição X
    }

    public int getY() {
        return (int) position.getY();  // Acesso à posição Y
    }

    public void setPosition(Position position) {
        this.position = position;  // Atualiza posição
    }

    // Método abstrato - deve ser implementado por subclasses
    public abstract void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGame
}
```

Código em Position.java - Classe de dados:

```
public class Position {
    private double x;
    private double y;
```

```
    public Position(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Position)) return false;
        Position position = (Position) o;
        if (Double.compare(position.x, x) != 0) return false;
        return Double.compare(position.y, y) == 0;
    }

    @Override
    public String toString() {
        return x + ", " + y;
    }
}
```

4.7 Sistema de Sprites e Enums

Código em SpritesBird.java - Enum para estados do bird:

```
public enum SpritesBird {
    BIRDY(new String[]{
            "****__",
            "___( O)>",
            "( <_. )",
            "*'___´"
    }),

    DEADBIRDY(new String[]{
            "****__",
            "___( X)>",  // Olho muda de O para X
            "( <_. )",
            "*'___´"
    });
```

```
    private final String[] sprite;

    SpritesBird(String[] sprite) {
        this.sprite = sprite;
    }

    public String[] getSprite() {
        return sprite;
    }
}
```

Código em SpritesGame.java - Enum para sprites do jogo:

```
public enum SpritesGame {
    TITLE(new String[]{
            " __  _                                _     _                _  ",
            "/  _| |                              | |   (_)              | | ",
            "| |_| | ___  _ __   _ __  _   _  ___ | |__  _ _ __  __| | ",
            "|  _| |/ _` |  _  ||  _  | | | | | | |_  | |  __/  _` | ",
            "| | | | (_| | |_) || |_) | |_| | | | |_) | | |  | (_| | ",
            "|_| |_||__,_ .__ / | .__/ |__, | |_.__/|_|_|   |__,_| ",
            "            | |    | |     __/ |                        ",
            "            |_|    |_|    |___/                         ",
    }),

    ENDING(new String[]{
            "  _______  _______  __   __  _______ ",
            " |       ||   _   ||  |_|  ||       | ",
            " |    ___||  |_|  ||       ||    ___|  ",
            " |   | __ |       ||       ||   |___   ",
            " |   ||  ||       ||       ||    ___| ",
            " |   |_| ||   _   || ||_|| ||   |___   ",
            " |_______||__| |__||_|   |_||_______|  ",
            "                                       ",
            "  _______  __   __  _______  ______    ",
            " |       ||  | |  ||       ||    _ |  ",
            " |   _   ||  |_|  ||    ___||   | ||  ",
            " |  | |  ||       ||   |___ |   |_||_ ",
            " |  |_|  ||       ||    ___||     __  |",
            " |       | |     | |   |___ |   |  | |",
            " |_______|  |___|  |_______||___|  |_|",
    });

    private final String[] sprite;

    SpritesGame(String[] sprite) {
```

```
        this.sprite = sprite;
    }

    public String[] getSprite() {
        return sprite;
    }
}
```

Uso dos enums no código:

Em Bird.java:

```
private SpritesBird birdState;

Bird(int x, int y) {
    super(x, y);
    birdState = SpritesBird.BIRDY;  // Estado inicial
}

public void setState(SpritesBird newState) {
    birdState = newState;  // Muda sprite quando morre
}

public void draw(...) {
    String[] draw = birdState.getSprite();  // Obtém sprite do enum atual
    // ... renderização
}
```

Em Arena.java:

```
// Muda sprite quando há colisão
bird.setState(SpritesBird.DEADBIRDY);

// Reseta sprite ao reiniciar
bird.setState(SpritesBird.BIRDY);
```

Em Game.java:

```
// Renderiza título do menu
String[] draw = SpritesGame.TITLE.getSprite();
for (int y = 0; y < draw.length; y++) {
    for (int x = 0; x < draw[y].length(); x++) {
        char c = draw[y].charAt(x);
        if (c != ' ') {
            graphics.putString(new TerminalPosition(titleX + x, titleY + y),
                               String.valueOf(c));
        }
    }
}
```

```java
// Renderiza "GAME OVER"
String[] draw = SpritesGame.ENDING.getSprite();
// ... mesmo processo
```

4.8 Herança e Polimorfismo

Hierarquia de herança:

```
GameComplements (abstract)
     > Bird
     > Pipe (também implements Scoring)
     > Wall
```

Código a demonstrar polimorfismo em Arena.java:

```java
// Lista heterogénea de GameComplements
private List<Wall> walls;  // Lista de Walls
private List<Pipe> pipes;  // Lista de Pipes

// Polimorfismo: todos implementam draw() de forma diferente
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
    // Itera sobre pipes - cada Pipe.draw() é chamado
    for (Pipe pipe : pipes) {
        pipe.draw(graphics, StartOfGameScreenX, StartOfGameScreenY);
    }

    // Itera sobre walls - cada Wall.draw() é chamado
    for (Wall wall : walls) {
        wall.draw(graphics, StartOfGameScreenX, StartOfGameScreenY);
    }

    // Bird também herda de GameComplements
    bird.draw(graphics, StartOfGameScreenX, StartOfGameScreenY, pipes, height);
}
```

Implementação de métodos abstratos:

Em Bird.java:

```java
@Override
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
    // Overload: versão com parâmetros adicionais
    draw(graphics, StartOfGameScreenX, StartOfGameScreenY, null, 0);
}

// Implementação específica do Bird
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY,
                 List<Pipe> pipes, int arenaHeight) {
```

```
    // Renderização complexa com background dinâmico
    // ...
}
```

Em Pipe.java:

```
@Override
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
    // Implementação específica do Pipe
    // Desenha pipe de cima e de baixo
    // ...
}
```

Em Wall.java:

```
@Override
public void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGameScreenY) {
    // Implementação específica do Wall
    // Desenha caractere simples (| ou -)
    // ...
}
```

5. Funcionalidades Planeadas Não Implementadas

- **Persistência de Best Score fora do jogo**: O best score não é salvo em ficheiro, é perdido após fechar o jogo
- **Diferentes Dificuldades**: Não há níveis de dificuldade (velocidade dos pipes, tamanho do gap) -**Sprites pixelizados**: cada carácter = píxel, tanto no pássaro como nos pipes
- **Mais sprites de bird**: um sprite para quando "salta"/voa e quando está a cair
- **Fullscreen**: um Fullscreen apropriado, onde o terminal seria colocado no centro do ecrã horizontalmente e na vertical totalmente esticado

6. Conceitos de Programação Orientada a Objetos e Boas Práticas

6.1 Conceitos de POO Utilizados

Herança

- **GameComplements**: Classe abstrata base para `Bird`, `Pipe` e `Wall`
  - Encapsula posicionamento comum
  - Define método abstrato `draw()` que deve ser implementado pelas subclasses

```
public abstract class GameComplements {
    protected Position position;
```

```
    public abstract void draw(TextGraphics graphics, int StartOfGameScreenX, int StartOfGame
}
```

Polimorfismo

- **Método `draw()`**: Cada classe filha implementa de forma diferente
- **Interface `Scoring`**: Permite que diferentes entidades implementem sistema de pontuação

Encapsulamento

- Atributos privados com métodos getters/setters
- Uso de `protected` para permitir acesso controlado em herança

Interfaces

- **Scoring**: Interface para sistema de pontuação, implementada por `Pipe`
  - Método default `checkAndScore()` demonstra uso de métodos default em interfaces

Enums

- **SpritesBird**: Enum para estados visuais do pássaro (BIRDY, DEADBIRDY)
- **SpritesGame**: Enum para sprites do jogo (TITLE, ENDING)

6.2 Boas Práticas

Separação de Responsabilidades

- **Game**: Gerencia estados e ciclo principal
- **Arena**: Lógica do jogo (física, colisões, pontuação)
- **Controller**: Processamento de input
- **SoundEffects**: Gestão de sons
- Cada classe tem uma responsabilidade clara

Composição sobre Herança

- `Arena` compõe `Bird`, `List<Pipe>`, `List<Wall>` em vez de herdar
- `SoundEffects` compõe múltiplos objetos `Sound`

Princípio DRY (Don't Repeat Yourself)

- Métodos reutilizáveis para cálculos comuns
- Enum para sprites evita duplicação de código

Naming Conventions

- Nomes descritivos e em inglês
- Convenções Java seguidas (camelCase para métodos/variáveis)

Tratamento de Exceções

- Try-catch para operações de I/O (sons, terminal)
- Mensagens de erro informativas

7. Descrição dos Testes

7.1 O que foi testado

ArenaTest.java

- **Colisão com parede inferior**: Verifica se bird morre ao tocar no chão (seria igual com os outros lados das paredes)
- **Colisão com pipe**: Verifica deteção de colisão com pipes
- **Sem colisão**: Verifica que bird não morre quando não há colisão
- **Sistema de pontuação**: Verifica que score aumenta após passar pipes

Código de exemplo:

```
@Test
void checkCollision_WithLowerWall() {
    arena.start();
    Bird bird = arena.getBird();
    bird.setPosition(new Position(20, 39));
    arena.checkCollision();
    assertTrue(arena.isBirdDead());
    assertEquals(SpritesBird.DEADBIRDY, bird.getState());
}
```

BirdTest.java

- **Estados do bird**: Verifica getState() e setState()
- **Posicionamento**: Testa getX(), getY(), setPosition()
- **Hitbox**: Testa getLeftX(), getRightX(), getStartY(), getEndY(), getHeight()

Código de exemplo:

```
@Test
public void setState_DeadBirdy() {
    assertEquals(SpritesBird.BIRDY, bird.getState());
    bird.setState(SpritesBird.DEADBIRDY);
    assertEquals(SpritesBird.DEADBIRDY, bird.getState());
}
```

PipeTest.java

- **Posição do pipe**: Verifica getX()
- **Movimentação**: Testa moveLeft() uma e múltiplas vezes
- **Sistema de scoring**: Verifica setAsScored() e isScored()
- **Gap**: Testa getGapStart() e getGap()

Código de exemplo:

```
@Test
public void moveLeft() {
    int initialX = pipe.getX();
    pipe.moveLeft();
    assertEquals(initialX - 1, pipe.getX());
}
```

7.2 O que não foi testado e porquê

Game.java

- **Não testado**: Classe principal com dependências de terminal
- **Razão**: Requer mock complexo de `Screen` e `Terminal` da biblioteca Lanterna, difícil de testar unitariamente

Controller.java

- **Não testado**: Thread de input
- **Razão**: Requer mock de `Screen` e teste de concorrência, complexo para testes unitários

Sound.java e SoundEffects.java

- **Não testado**: Reprodução de sons
- **Razão**: Depende de ficheiros de áudio e sistema de som do OS, melhor testado manualmente

GameComplements.java

- **Não testado**: Classe abstrata
- **Razão**: Testada indiretamente através das subclasses (Bird, Pipe, Wall)

Position.java

- **Não testado**: Classe simples de dados
- **Razão**: Funcionalidade básica, testada indiretamente através de outras classes

Integração Completa

- **Não testado**: Fluxo completo do jogo
- **Razão**: Requer testes de integração mais complexos que envolvem múltiplos componentes

8. Problemas Encontrados

8.1 Problemas Encontrados e Resolvidos

- **Resize do terminal**: Implementado sistema de redimensionamento dinâmico
- **Colisões precisas**: Sistema de hitbox implementado corretamente
- **Prevenção de duplicação de score**: Sistema de flags implementado

8.2 Problemas Encontrados e Não Resolvidos

Persistência de Best Score

- **Problema**: Best score não é salvo em ficheiro
- **Impacto**: Melhor pontuação é perdida ao fechar o jogo
- **Razão não resolvido**: Não foi prioridade na implementação inicial

###Resize do terminal é feita, mas não da maneira inicialmente planeada: - **Problema**: Jogo não faz resize 100% adequado em Fullscreen (intencionado inicialmente em colocar o jogo no centro e estendido verticalmente - estilo mobile no pc) - **Impacto**: jogo centraliza mas não estica ao máximo verticalmente - **Razão não resolvido**: teria de ser feito uma escala para aumentar tudo de forma proporcional + aumentar ASCII dos sprites do bird de alguma maneira

9. Possíveis Melhorias

Arquitetura

1. **Padrão State**: Implementar padrão State para gerir estados do jogo (Menu, Playing, GameOver)
2. **Factory Pattern**: Para criação de pipes com diferentes configurações
3. **Observer Pattern**: Para sistema de eventos (colisão, pontuação)

Funcionalidades

1. **Persistência**: Salvar best score em ficheiro JSON/XML
2. **Configurações**: Menu de opções (volume, cores, dificuldade)
3. **Animations**: Animações suaves para transições
4. **Power-ups**: Itens especiais para aumentar score ou facilitar jogo

Código

1. **Constantes**: Mover valores mágicos para constantes nomeadas

2. **Documentação**: Adicionar JavaDoc a métodos públicos
3. **Refatoração**: Extrair métodos longos em métodos menores
4. **Testes**: Aumentar cobertura de testes (especialmente integração)

Performance

1. **Object Pooling**: Reutilizar objetos Pipe em vez de criar/destruir
2. **Otimização de renderização**: Reduzir chamadas desnecessárias de draw

10. Outros Aspetos Relevantes

10.1 Design Patterns Utilizados

- **Template Method**: `GameComplements.draw()` define estrutura, subclasses implementam
- **Strategy (parcial)**: Interface `Scoring` permite diferentes estratégias de pontuação
- **Composition**: Uso extensivo de composição sobre herança

10.2 Gestão de Memória

- **Listas dinâmicas**: Pipes são removidos quando saem do ecrã, evitando memory leaks
- **Thread management**: Controller thread é gerida corretamente com flag `finish`

10.3 Compatibilidade

- **Java 8**: Código compatível com Java 8 (Amazon Corretto)
- **Cross-platform**: Funciona em Windows, Linux, macOS (dependendo da Lanterna)

10.4 Experiência do Utilizador

- **Feedback visual**: Sprites ASCII bem desenhados
- **Feedback sonoro**: Sons para todas as ações importantes
- **Interface intuitiva**: Controles simples (espaço, Q, E)

10.5 UML Diagrams

Importante: a imagem no github não permite uma boa visualização. Para ver melhor acesse ou o jpg presente nesta pasta ou, se não tiver uma visualização, ainda assim, suficiente, pode acessar o link a seguir:
Flappy Bird UML - Miro
Este link redireciona para o site onde o uml foi criado, onde pode

visualizar com qualidade e com o *zoom* necessário.
flappybird_uml

11. Conclusões

Este projeto demonstra uma implementação sólida do jogo Flappy Bird em Java, utilizando programação orientada a objetos de forma eficaz. O código está bem estruturado, com ajuda da biblioteca Lanterna, com uma separação clara de responsabilidades e uso adequado de conceitos de POO como herança, interfaces, polimorfismo e threads. Assim, o projeto cumpre os objetivos principais (de implementação).

12. Bibliografia e Recursos Utilizados

12.1 Bibliotecas e Dependências

- **Lanterna - documentação da biblioteca**: https://github.com/mabe02/lanterna
- **JUnit Jupiter**: para os testes
- **Maven**: dependências

12.2 Recursos Online

- **Stack Overflow**: para resolução de problemas e implementação de determinados códigos
- **Oracle Java Tutorials**: para implementação de determinados códigos (como o sound, etc)
- **Vídeos online de implementação**: como https://www.youtube.com/watch?v=SyZQVJiARTQ e https://youtu.be/qPVkRtuf9CQ

13. Auto-Avaliação

13.1 Contribuição de Cada Membro

Felipe Castilho (a047152) - 33%

Juliana Moreira (a047188) - 33%

Marta Vieira (a046756) - 33%